

# Threat Modelling

Lecture notes

## Goal of the chapter

This chapter gives you an introduction to threat modelling. The goal is to give you basic tools to draw flow diagrams for software systems, delineate trust boundaries, find out what can go wrong by using the STRIDE model, and coming out with basic mitigation strategies.

## Acknowledgements

The content on threat modelling is based on the book mostly based on the book *Threat Modeling: Designing for Security*, John Wiley Sons, Incorporated, 2014, by Adam Shostack.

## 1 What is threat modelling?

It is safe to say that software is everywhere, and it needs to be secure. Security can be hard to achieve, but many methodologies for securing systems exist today. All try to answer the same fundamental set of questions.

- What is being protected?
- Are there any known threats and vulnerabilities?
- What are the impacts to the organization if the data is lost or leaked?
- What is the value of the data to the organization?
- What can be done to mitigate the risks?

Threat modelling is about identifying all potential threats to a system, document them, prioritise them, and address them. Asset, threat, vulnerability, and exploit are the most commonly-used terms in the cyber security lingo. Depending on the literature some of these terms overlap and are sometimes used synonymously.

- An asset is what is being protected, something that has some value to its owner. Its value can be tangible (e.g. gold or a running server) or intangible (data)
- A threat is an intention to cause damage. For cyber security this can be defined as a hostile act aimed by an attacker at an asset. Regardless of the attacker's intent to do no harm, a threat is still a threat. The attacker posing a threat is commonly called a threat actor.
- A vulnerability is a defect in the target system. This defect may be a bug in application code, or a flaw in the design of the system. Vulnerabilities can also be a consequence of improper configuration or user action.
- An exploit is a way to take advantage of a known vulnerability. The usual objective is to take control over the asset. (Social engineering, commonly considered a simple scam, is one kind of exploit.)

Threats can be revealed by a variety of actors. For example, an ordinary user may stumble on a flaw in an application; a script kiddie running automated tools may discover a flaw; or a truly motivated attacker may find a flaw in the application through manual analysis. A threat's impact on an application might include unauthorized access being granted due to authorization failure, the browser cache being poisoned with malicious data, or private data being revealed via eavesdropping.

To simplify modelling, multiple ways exist of classifying threats. Examples are STRIDE, DREAD and Attack Trees. Neither one is exhaustive, but they provide good structures for determining the type of a given threat.

## 2 Learning to Threat Model

You begin threat modeling by focusing on four key questions:

- What are you building?
- What can go wrong?
- What should you do about those things that can go wrong?
- Did you do a decent job of analysis?

In addressing these questions, you start and end with tasks that all technologists should be familiar with: drawing on a whiteboard and managing bugs. In between, this chapter will introduce a variety of new techniques you can use to think about threats. If you get confused, just come back to these four questions. Everything in this chapter is designed to help you answer one of these questions. You're going to first walk through these questions using a three-tier web app as an example, and after you've read that, you should walk through the steps again with something of your own to threat model. It could be software you're

building or deploying, or software you're considering acquiring. If you're feeling uncertain about what to model, you can browse for sample or real-life systems, for example, at:

- <https://docs.microsoft.com/en-us/biztalk/core/sample-scenarios-for-threat-model-analysis>
- Appendix E of the book *Threat Modeling: Designing for Security* by Adam Shostack

### 3 What are you building?

Diagrams are a good way to communicate what you are building. There are lots of ways to diagram software, and you can start with a whiteboard diagram of how data flows through the system. In this example, we will be working with a simple web app with a web browser, web server, some business logic and a database.

#### Running example

As a running example and recurrent goal throughout this course we will attempt to develop a web application that does the following.

- A lecturer submits questions and answers to a repository
- Questions should have annotations defining their area of knowledge.
- A student creates mock exams on a particular area of knowledge based on existing questions.

By no means we intend to show all steps of a Secure Software Life Cycle in a rigorous way. Instead, you will use it as a starting point to threat model your own system (see Figure 1).

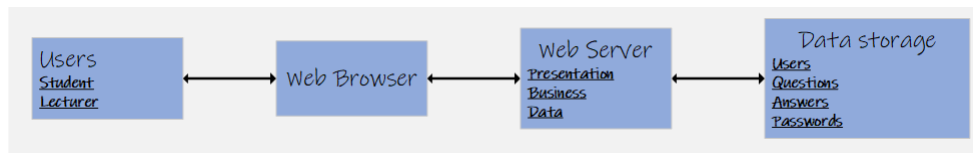


Figure 1: A simple Data Flow Diagram for our web application.

### Practical activity

Some people will actually start thinking about what goes wrong right here. For example, how do you know that the web browser is being used by the person you expect? What happens if someone modifies data in the database? Is it OK for information to move from one box to the next without being encrypted? You might want to take a minute to think about some things that could go wrong here because these sorts of questions may lead you to ask “is that allowed?”

Adding boundaries to show who controls what is a simple way to improve the diagram. You can pretty easily see that the threats that cross those boundaries are likely important ones, and may be a good place to start identifying threats. These boundaries are called trust boundaries, and you should draw them wherever different people control different things. Good examples of this include the following:

- Accounts (UIDs on unix systems, or SIDS on Windows)
- Network interfaces
- Different physical computers
- Virtual machines
- Organizational boundaries
- Almost anywhere you can argue for different privileges

### Trust boundary vs Attack surface

A closely related concept that you may have encountered is attack surface. For example, the hull of a ship is an attack surface for a torpedo. The side of a ship presents a larger attack surface to a submarine than the bow of the same ship. The ship may have internal “trust” boundaries, such as waterproof bulkheads or a Captain’s safe. A system that exposes lots of interfaces presents a larger attack surface than one that presents few APIs or other interfaces. Network firewalls are useful boundaries because they reduce the attack surface relative to an external attacker. However, much like the Captain’s safe, there are still trust boundaries inside the firewall. A trust boundary and an attack surface are very similar views of the same thing. An attack surface is a trust boundary and a direction from which an attacker could launch an attack. Many people will treat the terms as interchangeable.

We shall draw trust boundaries as boxes (see Figure 2), showing what’s inside each with a label (such as “corporate data center”) near the edge of the box.

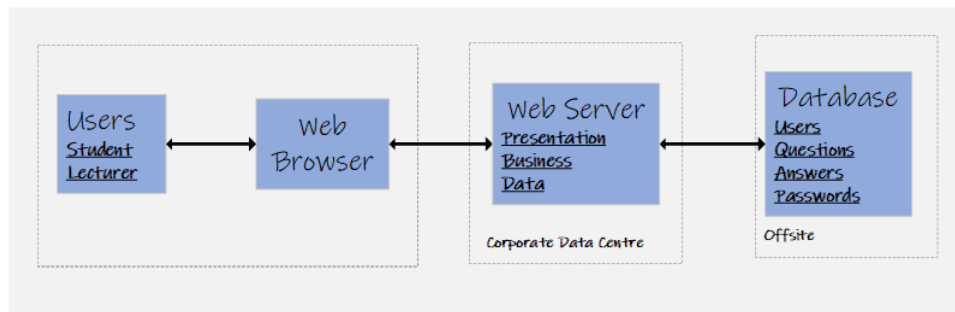


Figure 2: A simple Data Flow Diagram for our web application with trust boundaries.

Regarding the physical form of the diagram: Use whatever works for you. If that's a whiteboard diagram and a camera phone picture, great. If it's Visio, or OmniGraffle, or some other drawing program, great. You should think of threat model diagrams as part of the development process, so try to keep it in source control with everything else.

#### Research activity

Your task is to describe and explain the *Principle of minimising attack surface area and securing the weakest link*. Find real-life examples of vulnerabilities that are due to the violation of that principle.

## 4 What can go wrong?

Now that you have a diagram, you can really start looking for what can go wrong with its security. For instance, how do you know that the web browser is being used by the person you expect? What happens if someone modifies data in the database? Is it OK for information to move from one box to the next without being encrypted? You don't need to come up with these questions by just staring at the diagram and scratching your chin. You can identify threats like these using the simple mnemonic STRIDE, described in detail in the next section.

## 5 The STRIDE Threat Model

The STRIDE Threat Model is a useful checklist of questions that can help in the threat-modelling of an application. 'STRIDE' is an acronym for the following threat categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. Spoofing covers cases where someone is illegally accessing a system using another user's authentication informa-

tion. Tampering covers cases such as unauthorized changes made to persistent data, whether inside a machine or in the transport. Repudiation specifies that a system should be able to trace user operations to provide evidence of what has happened in case of a breach. Information Disclosure covers the exposure of information to unauthorized individuals. (This category of threat can also occur within a machine or during transport.) Denial of Service refers to cases where the server or service is made temporarily unavailable. Lastly, Elevation of Privilege is a threat type in which an unprivileged user finds a way to gain sufficient privileges to compromise the system.

Recall the three example threats mentioned in the preceding section:

- How do you know that the web browser is being used by the person you expect?
- What happens if someone modifies data in the database?
- Is it ok for information to go from one box to the next without being encrypted?

These are examples of spoofing, tampering, and information disclosure. Using STRIDE as a mnemonic can help you walk through a diagram and select example threats. Pair that with a little knowledge of security and the right techniques, and you'll find the important threats faster and more reliably. If you have a process in place for ensuring that you develop a threat model, document it, and you can increase confidence in your software.

#### Brainstorm activity

Now that you have STRIDE in your tool belt, walk through your diagram again and look for more threats, this time using the mnemonic. Make a list as you go with the threat and what element of the diagram it affects.

The following list provides some examples of each threat.

- **Spoofing:** Someone might pretend to be a lecturer and attempt to insert fake questions, so you'll need a way to authenticate lecturers. Someone might also pretend to be the web application, and provide students with mock exams that are incorrect, so you should ensure that you have an SSL certificate and that you use a single domain for all your pages (to help that subset of customers who read URLs to see if they're in the right place).
- **Tampering:** Someone might tamper with the data in your back end. Someone might tamper with the data as it flows back and forth.
- **Repudiation:** Any of the preceding actions might require digging into what happened. Are there system logs? Is the right information being logged effectively? Are the logs protected against tampering?

- **Information Disclosure:** Can anyone connect to the database and read or write information?
- **Denial of Service:** What happens if a thousand customers show up at once at the website?
- **Elevation of Privilege:** Perhaps the web front end is the only place customers should access, but what enforces that? What prevents them from connecting directly to the business logic server, or uploading new code? What controls access to your database, or what happens if an employee makes a mistake, or even wants to edit your database?

## 6 Addressing Each Threat

You should now have a decent-sized list or lists of threats. The next step in the threat modeling process is to go through the lists and address each threat.

The strategies listed in the following tables are intended to serve as examples to illustrate ways to address threats. Your “go-to” approach should be to mitigate threats. Mitigation is generally the easiest and the best for your customers. (It might look like accepting risk is easier, but over time, mitigation is easier.) Mitigating threats can be hard work, and you shouldn’t take these examples as complete. There are often other valid ways to address each of these threats, and sometimes trade-offs must be made in the way the threats are addressed.

### 6.1 Addressing Spoofing

Table 1 and the list that follows show targets of spoofing, mitigation strategies that address spoofing, and techniques to implement those mitigations.

- When you’re concerned about a person being spoofed, ensure that each person has a unique username and some way of authenticating. The traditional way to do this is with passwords, which have all sorts of problems as well as all sorts of advantages that are hard to replicate.
- When accessing a file on disk, don’t ask for the file with `open(file)`. Use `open(/path/to/file)`. If the file is sensitive, after opening, check various security elements of the file descriptor (such as fully resolved name, permissions, and owner). You want to check with the file descriptor to avoid *race conditions*. This applies doubly when the file is an executable, although checking after opening can be tricky. Therefore, it may help to ensure that the permissions on the executable can’t be changed by an attacker.
- When you’re concerned about a system or computer being spoofed when it connects over a network, you’ll want to use DNSSEC, SSL, IPsec, or a combination of those to ensure you’re connecting to the right place.

Table 1: Addressing Spoofing Threats.

| Threat Target              |      | Mitigation Strategy                                                           | Mitigation Technique                                                                                            |
|----------------------------|------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Spoofing person            | a    | Identification and authentication (usernames and something you know/have/are) | Usernames, real names, or other identifiers: Passwords, Tokens and Biometrics.<br>Enrollment/maintenance/expiry |
| Spoofing 'file' on disk    | a    | Leverage the OS<br><br>Cryptographic authenticators                           | Full paths, Checking ACLs, Ensuring that pipes are created properly<br>Digital signatures or authenticators     |
| Spoofing a network address |      | Cryptographic                                                                 | DNSSEC, HTTPS/SSL, IPsec                                                                                        |
| Spoofing program in memory | a in | Leverage the OS                                                               | Many modern operating systems have some form of application identifier that the OS will enforce.                |

## 6.2 Addressing Tampering

Table 2 and the list that follows show targets of tampering, mitigation strategies that address tampering, and techniques to implement those mitigations.

- **Tampering with a file:** Tampering with files can be easy if the attacker has an account on the same machine, or by tampering with the network when the files are obtained from a server.
- **Tampering with memory:** The threats you want to worry about are those that can occur when a process with less privileges than you, or that you don't trust, can alter memory. For example, if you're getting data from a shared memory segment, is it ACLed so only the other process can see it? For a web app that has data coming in via AJAX, make sure you validate that the data is what you expect after you pull in the right amount.
- **Tampering with network data:** Preventing tampering with network data requires dealing with both spoofing and tampering. Otherwise, someone who wants to tamper can simply pretend to be the other end, using what's called a man-in-the-middle attack. The most common solution to these problems is SSL, with IP Security (IPsec) emerging as another possibility. SSL and IPsec both address confidentiality and tampering, and can help address spoofing.



- **Tampering with networks anti-pattern:** It’s somewhat common for people to hope that they can isolate their network, and so not worry about tampering threats. It’s also very hard to maintain isolation over time. Isolation doesn’t work as well as you would hope.

Table 2: Addressing Tampering Threats.

| Threat Target                                            | Mitigation Strategy                                              | Mitigation Technique                                                                                                  |
|----------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Tampering with a file                                    | Operating system                                                 | ACLs.                                                                                                                 |
|                                                          | Cryptographic                                                    | Digital Signatures and Keyed MAC                                                                                      |
| Racing to create a file (tampering with the file system) | Using a directory that’s protected from arbitrary user tampering | ACLs. Using private directory structures. (Randomizing your file names just makes it annoying to execute the attack.) |
| Tampering with a network packet                          | Cryptographic                                                    | HTTPS/SSL, IPsec                                                                                                      |
|                                                          | Anti-pattern                                                     | Network isolation (See note on network isolation anti-pattern.)                                                       |

### 6.3 Addressing Repudiation

Addressing repudiation is generally a matter of ensuring that your system is designed to log and ensuring that those logs are preserved and protected. Some of that can be handled with simple steps such as using a reliable transport for logs. In this sense, syslog over UDP was almost always silly from a security perspective; syslog over TCP/SSL is now available and is vastly better.

Table 3 and the list that follows show targets of repudiation, mitigation strategies that address repudiation, and techniques to implement those mitigations.

- **No logs means you can’t prove anything:** This is self-explanatory. For example, when a customer calls to complain that they never got their order, how will this be resolved? Maintain logs so that you can investigate what happens when someone attempts to repudiate something.
- **Logs come under attack:** Attackers will do things to prevent your logs from being useful, including filling up the log to make it hard to find the attack or forcing logs to “roll over.” They may also do things to set off

so many alarms that the real attack is lost in a sea of troubles. Perhaps obviously, sending logs over a network exposes them to other threats that you'll need to handle.

- **Logs as a channel for attack:** By design, you're collecting data from sources outside your control, and delivering that data to people and systems with security privileges. An example of such an attack might be sending mail addressed to `</html> haha@example.com`, causing trouble for web-based tools that don't expect inline HTML.

Table 3: Addressing Repudiation Threats.

| Threat Target                           | Mitigation Strategy                                   | Mitigation Technique                                    |
|-----------------------------------------|-------------------------------------------------------|---------------------------------------------------------|
| No logs means you can't prove anything. | Be sure to log all the security-relevant information. | .                                                       |
| Logs come under attack                  | Protect your logs.                                    | Send over the network. ACL.                             |
| Logs as a channel for attack            | Tightly specified logs                                | Documenting log design early in the development process |

You can make it easier to write secure code to process your logs by clearly communicating what your logs can't contain, such as "Our logs are all plaintext, and attackers can insert all sorts of things," or "Fields 1–5 of our logs are tightly controlled by our software, fields 6–9 are easy to inject data into. Field 1 is time in GMT. Fields 2 and 3 are IP addresses (v4 or 6)..." Unless you have incredibly strict control, documenting what your logs can contain will likely miss things. (For example, can your logs contain Unicode double-wide characters?)

## 6.4 Addressing information disclosure

Table 4 and the list which follows show targets of information disclosure, mitigation strategies that address information disclosure, and techniques to implement those mitigations.

- **Network monitoring:** Network monitoring takes advantage of the architecture of most networks to monitor traffic. (In particular, most networks now broadcast packets, and each listener is expected to decide if the packet matters to them.) When networks are architected differently, there are a variety of techniques to draw traffic to or through the monitoring station. If you don't address spoofing, much like tampering, an attacker can just

sit in the middle and spoof each end. Mitigating network information disclosure threats requires handling both spoofing and tampering threats. If you don't address tampering, then there are all sorts of clever ways to get information out. Here again, SSL and IP Security options are your simplest choices.

- **Names reveal information:** When the name of a directory or a filename itself will reveal information, then the best way to protect it is to create a parent directory with an innocuous name and use operating system ACLs or permissions.
- **File content is sensitive:** When the contents of the file need protection, use ACLs or cryptography. If you want to protect all the data should the machine fall into unauthorized hands, you'll need to use cryptography. The forms of cryptography that require the person to manually enter a key or passphrase are more secure and less convenient. There's file, filesystem, and database cryptography, depending on what you need to protect.
- **APIs reveal information:** When designing an API, or otherwise passing information over a trust boundary, select carefully what information you disclose. You should assume that the information you provide will be passed on to others, so be selective about what you provide. For example, website errors that reveal the username and password to a database are a common form of this flaw, others are discussed in Chapter 3.

Table 4: Addressing Information Disclosure Threats.

| Threat Target                                                        | Mitigation Strategy              | Mitigation Technique                                                        |
|----------------------------------------------------------------------|----------------------------------|-----------------------------------------------------------------------------|
| Network monitoring                                                   | Encryption                       | HTTPS/SSL                                                                   |
| Directory or filename (for example layoff-letters/adamshostack.docx) | Leverage the OS.                 | ACLs                                                                        |
| File contents                                                        | Leverage the OS.<br>Cryptography | ACLs<br>File encryption such as PGP, disk encryption (FileVault, BitLocker) |
| API in-formation disclosure                                          | Design                           | Careful design control Consider pass by reference or value.                 |

## 6.5 Addressing Denial of Service

Table 5 and the list that follows show targets of denial of service, mitigation strategies that address denial of service, and techniques to implement those mitigations.

- **Network flooding:** If you have static structures for the number of connections, what happens if those fill up? Similarly, to the extent that it's under your control, don't accept a small amount of network data from a possibly spoofed address and return a lot of data. Lastly, firewalls can provide a layer of network ACLs to control where you'll accept (or send) traffic, and can be useful in mitigating network denial-of-service attacks.
- **Look for exhaustible resources:** The first set of exhaustible resources are network related, the second set are those your code manages, and the third are those the OS manages. In each case, elastic resourcing is a valuable technique. For example, in the 1990s some TCP stacks had a hardcoded limit of five half-open TCP connections. (A half-open connection is one in the process of being opened. Don't worry if that doesn't make sense, but rather ask yourself why the code would be limited to five of them.) Today, you can often obtain elastic resourcing of various types from cloud providers.
- **System resources:** Operating systems tend to have limits or quotas to control the resource consumption of user-level code. Consider those resources that the operating system manages, such as memory or disk usage. If your code runs on dedicated servers, it may be sensible to allow it to chew up the entire machine. Be careful if you unlimit your code, and be sure to document what you're doing.
- **Program resources:** Consider resources that your program manages itself. Also, consider whether the attacker can make you do more work than they're doing. For example, if he sends you a packet full of random data and you do expensive cryptographic operations on it, then your vulnerability to denial of service will be higher than if you make him do the cryptography first. Of course, in an age of botnets, there are limits to how well one can reassign this work. There's an excellent paper by Ben Laurie and Richard Clayton, "Proof of work proves not to work," which argues against proof of work schemes (Laurie, 2004).

## 6.6 Addressing Elevation of Privilege

Table 6 and the list that follows show targets of elevation of privilege, mitigation strategies that address elevation of privilege, and techniques to implement those mitigations.

Table 5: Addressing Denial Of Service Threats.

| Threat Target      | Mitigation Strategy                  | Mitigation Technique                                                                                                                                                                                                                                                                                                                                    |
|--------------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Network Flooding   | Look for exhaustible resources.      | Elastic resources. Work to ensure attacker resource consumption is as high as or higher than yours.<br>Network ACLS                                                                                                                                                                                                                                     |
| Program re-sources | Careful design<br>Avoid multipliers. | Elastic resource management, proof of work<br>Look for places where attackers can multiply CPU consumption on your end with minimal effort on their end: Do something to require work or enable distinguishing attackers, such as client does crypto first or login before large work factors (of course, that can't mean that logins are unencrypted). |
| System re-sources  | Leverage the OS.                     | Leverage the OS.                                                                                                                                                                                                                                                                                                                                        |

- **Data/code confusion:** Problems where data is treated as code are common. As information crosses layers, what's tainted and what's pure can be lost. Attacks such as XSS take advantage of HTML's freely interweaving code and data. (That is, an .html file contains both code, such as Javascript, and data, such as text, to be displayed and sometimes formatting instructions for that text.) There are a few strategies for dealing with this. The first is to look for ways in which frameworks help you keep code and data separate. For example, prepared statements in SQL tell the database what statements to expect, and where the data will be.

- You can also look at the data you're passing right before you pass it, so you know what validation you might be expected to perform for the function you're calling. For example, if you're sending data to a web page, you might ensure that it contains no <, >, #, or & characters, or whatever.
- In fact, the value of "whatever" is highly dependent on exactly what exists between "you" and the rendition of the web page, and what security checks it may be performing. If "you" means a web server, it may be very important to have a few j and i symbols in what you produce. If "you" is something taking data from a database and

sending it to, say PHP, then the story is quite different. Ideally, the nature of “you” and the additional steps are clear in your diagrams.

- **Control flow/memory corruption attacks:** This set of attacks generally takes advantage of weak typing and static structures in C-like languages to enable an attacker to provide code and then jump to that code. If you use a type-safe language, such as Java or C#, many of these attacks are harder to execute.
  - Modern operating systems tend to contain memory protection and randomization features, such as Address Space Layout Randomization (ASLR). Sometimes the features are optional, and require a compiler or linker switch. In many cases, such features are almost free to use, and you should at least try all such features your OS supports. (It’s not completely effortless, you may need to recompile, test, or make other such small investments.)
  - The last set of controls to address memory corruption are sandboxes. Sandboxes are OS features that are designed to protect the OS or the rest of the programs running as the user from a corrupted program.
- **Command injection attacks:** Command injection attacks are a form of code/data confusion where an attacker supplies a control character, followed by commands. For example, in SQL injection, a single quote will often close a dynamic SQL statement; and when dealing with unix shell scripts, the shell can interpret a semicolon as the end of input, taking anything after that as a command.

## 7 File Bugs

Now that you have a list of threats and ways you would like to mitigate them, you’re through the complex, security-centered parts of the process. There are just a few more things to do, the first of which is to treat each line of the preceding tables as a bug. You want to treat these as bugs because if you ship software, you’ve learned to handle bugs in some way. You presumably have a way to track them, prioritize them, and ensure that you’re closing them with an appropriate degree of consistency. This will mean something very different to a three-person start-up versus a medical device manufacturer, but both organizations will have a way to handle bugs. You want to tap into that procedure to ensure that threat modeling isn’t just a paper exercise.

You can write the text of the bugs in a variety of ways, based on what your organization does. Examples of filing a bug might include the following:

- Someone might use the `/admin/` interface without proper authorization.
- The admin interface lacks proper authorization controls,

- There's no automated security testing for the `/admin/` interface.

Whichever way you go, it's great if you can include the entire threat in the bug, and mark it as a security bug if your bug-tracking tool supports that. (If you're a super-agile scrum shop, use a uniquely colored Post-it for security bugs.)

You'll also have to prioritize the bugs. Elevation-of-privilege bugs are almost always going to fall into the highest priority category, because when they're exploited they lead to so much damage. Denial of service often falls toward the bottom of the stack, but you'll have to consider each bug to determine how to rank it.

#### Research activity

Find out the differences between the DREAD model and the STRIDE model.

## 8 Checking Your Work

Validation of your threat model is the last thing you do as part of threat modeling. There are a few tasks to be done here, and it is best to keep them aligned with the order in which you did the previous work. Therefore, the validation tasks include checking the model, checking that you've looked for each threat, and checking your tests. You probably also want to validate the model a second time as you get close to shipping or deploying.

### 8.1 Checking the model

You should ensure that the final model matched what you built. If it doesn't, how can you know that you found the right, relevant threats? To do so, try to arrange a meeting during which everyone looks at the diagram, and answer the following questions:

- Is this complete?
- Is it accurate?
- Does it cover all the security decisions we made?
- Can I start the next version with this diagram without any changes?

If everyone says yes, your diagram is sufficiently up to date for the next step. If not, you'll need to update it.

## 8.2 Updating the Diagram

As you went through the diagram, you might have noticed that it's missing key data. If it were a real system, there might be extra interfaces that were not drawn in, or there might be additional databases. There might be details that you jumped to the whiteboard to draw in. If so, you need to update the diagram with those details. A few rules of thumb are useful as you create or update diagrams:

- Focus on data flow, not control flow.
- Anytime you need to qualify your answer with “sometimes” or “also,” you should consider adding more detail to break out the various cases. For example, if you say, “Sometimes we connect to this web service via SSL, and sometimes we fall back to HTTP,” you should draw both of those data flows (and consider whether an attacker can make you fall back like that).
- Anytime you find yourself needing more detail to explain security-relevant behavior, draw it in.
- Any place you argued over the design or construction of the system, draw in the agreed-on facts. This is an important way to ensure that everyone ended that discussion on the same page. It's especially important for larger teams when not everyone is in the room for the threat model discussions. If they see a diagram that contradicts their thinking, they can either accept it or challenge the assumptions; but either way, a good clear diagram can help get everyone on the same page.
- Don't have data sinks: You write the data for a reason. Show who uses it.
- Data can't move itself from one data store to another: Show the process that moves it.
- The diagram should tell a story, and support you telling stories while pointing at it.
- Don't draw an eye chart (a diagram with so much detail that you need to squint to read the tiny print).

## 8.3 Diagram Details

If you're wondering how to reconcile that last rule of thumb, don't draw an eye chart, with all the details that a real software project can entail, one technique is to use a sub diagram that shows the details of one particular area. You should look for ways to break things out that make sense for your project. For example, if you have one hyper-complex process, maybe everything in that process should be covered in one diagram, and everything outside it in another. If you have



a dispatcher or queuing system, that's a good place to break things up. Your databases or the fail-over system is also a good split. Maybe there's a set of a few elements that really need more detail. All of these are good ways to break things out.

The key thing to remember is that the diagram is intended to help ensure that you understand and can discuss the system. Recall the quote that opens this book: "All models are wrong. Some models are useful." Therefore, when you're adding additional diagrams, don't ask, "Is this the right way to do it?" Instead, ask, "Does this help me think about what might go wrong?"

### Threat Modeling on Your Own

You have now walked through your first threat model. Congratulations! Remember though: You're not going to get to Carnegie Hall if you don't practice, practice, practice. That means it is time to do it again, this time on your own, because doing it again is the only way to get better. Pick a system you're working on and threat model it. Follow this simplified, five-step process as you go:

- Draw a diagram.
- Use the EoP game to find threats.
- Address each threat in some way.
- Check your work with the checklists at the end of this chapter.

Right now, if you're new to threat modeling, your best bet is to do it often, applying it to the software and systems that matters to you. After threat modeling a few systems, you'll find yourself getting more comfortable with the tools and techniques. For now, the thing to do is practice. Build your first muscles to threat model with.

## 9 Checklists for Diving In and Threat Modeling

Here are some checklists that are designed to help you avoid the most common problems. Each question is designed to be read aloud and to have an affirmative answer from everyone present. After reading each question out loud, encourage questions or clarification from everyone else involved.

### 9.1 Diagramming

1. Can we tell a story without changing the diagram?
2. Can we tell that story without using words such as "sometimes" or "also"?
3. Can we look at the diagram and see exactly where the software will make a security decision?

4. Does the diagram show all the trust boundaries, such as where different accounts interact? Do you cover all UIDs, all application roles, and all network interfaces?
5. Does the diagram reflect the current or planned reality of the software?
6. Can we see where all the data goes and who uses it?
7. Do we see the processes that move data from one data store to another?

## 9.2 Threats

1. Have we looked for each of the STRIDE threats?
2. Have we looked at each element of the diagram?
3. Have we looked at each data flow in the diagram?

## 9.3 Validating Threats

1. Have we written down or filed a bug for each threat?
2. Is there a proposed/planned/implemented way to address each threat?
3. Do we have a test case per threat?
4. Has the software passed the test?

### Research activity

Find out what are abuse/security cases and abuse/security stories. When looking at a system from the perspective of an attacker, investigate how useful attack trees are.

Table 6: Addressing Elevation of Privileges Threats.

| Threat Target                          | Mitigation Strategy                                      | Mitigation Technique                                                                                                                                                                                                                                                                                   |
|----------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data/code confusion                    | Use tools and architectures that separate data and code. | Prepared statements or stored procedures in SQL. Clear separators with canonical forms. Late validation that data is what the next function expects.                                                                                                                                                   |
| Control flow/memory corruption attacks | Use a type-safe language.                                | Writing code in a type-safe language protects against entire classes of attack.                                                                                                                                                                                                                        |
|                                        | Leverage the OS for memory protection.                   | Most modern operating systems have memory-protection facilities.                                                                                                                                                                                                                                       |
|                                        | Use the sandbox.                                         | Modern operating systems support sandboxing in various ways (AppArmor on Linux, AppContainer or the MOICE pattern on Windows, Sandboxlib on Mac OS). Don't run as the "nobody" account, create a new one for each app. Postfix and QMail are examples of the good pattern of one account per function. |
| Command injection attacks              | Be careful.                                              | Validate that your input is the size and form you expect. Don't sanitize. Log and then throw it away if it's weird.                                                                                                                                                                                    |